DocChain: Complete Presentation Deck (Review Ready)

This document contains the exact slide-by-slide text, tables, and architecture diagram matching the format of your reference presentation.

Slide 1: Title Slide

DOCCHAIN: Secure Document Issuance and Verification System

Introduction to Cloud Computing – Group 18

- **CB.SC.U4AIE24131** – M. Sarveshwara Reddy
- **CB.SC.U4AIE24144** – Varshitha P
- **CB.SC.U4AIE24146** – Puneeth Kumar Kalinga
- **CB.SC.U4AIE24150** – S. Tagur Misra

Slide 2: Introduction

- The DocChain system manages official documents, user accounts, verification sessions, and security access logs across a document lifecycle.
- The system provides essential document operations, including PDF issuance, dynamic QR code stamping, cryptographic hashing, and real-time public verification.
- Cloud computing enables scalable, reliable, and accessible infrastructure for hosting document databases, encrypted storage, and verification APIs.
- The project uses Microsoft Azure cloud services to support application deployment, database connectivity, and secure access.
- The final implementation integrates the frontend, FastAPI backend, and cloud database to provide a functional secure document management solution.

Slide 3: Problem Statement

Existing challenges:

- Managing large amounts of academic and institutional document records
- Maintaining document integrity and preventing digital forgery (altered grades/certificates)
- Securely handling sensitive PDF files and preventing privacy leaks
- Providing instant, automated verification without slow manual email checks
- Scaling the system reliably as verification requests increase
- Managing database availability, access control, and revocation of expired documents

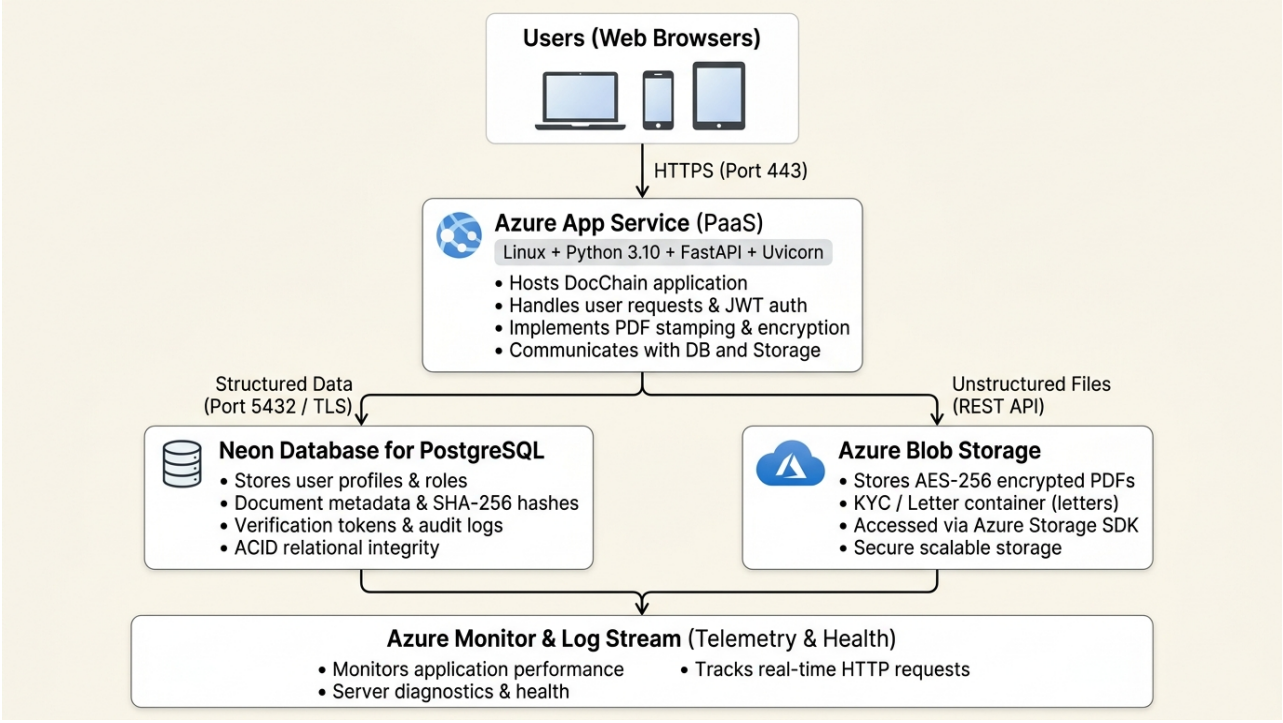
Slide 4: Objectives

- Design a relational database to manage users, roles, issued documents, and verification tokens.
- Implement secure document operations, including dynamic QR code stamping, SHA-256 hashing, and AES-256 encryption.
- Maintain a tamper-proof audit trail of access logs to monitor issuance, downloads, and verification events.
- Implement user authentication, role-based authorization (Admin, Staff, Recipient), and session security using JWT tokens.
- Develop a backend REST API to integrate the frontend dashboards with cloud storage and database services.

Slide 5: Scope of the system

USER	ROLE
Recipient (Student)	Views received documents; downloads decrypted PDFs with verified stamps; checks document validity.
Staff (Issuer)	Uploads official PDF documents for students; dynamically generates QR codes; revokes issued letters when needed.
Admin	Manages system users and roles; activates or suspends accounts; oversees audit logs and system security.
Verifier (Public / Employer)	Scans QR codes via camera or uploads document images to instantly verify legitimacy and active status.

Slide 6: Architecture of the system



• **Users (Web Browsers):** Clients access the system over HTTPS (Port 443).

• **Azure App Service (PaaS):** Hosts the Linux + Python 3.10 + FastAPI application, handles business logic, and coordinates stamping and cryptography.

• **Neon PostgreSQL:** Stores relational user data, document metadata, SHA-256 signatures, and access logs over SSL (Port 5432).

• **Azure Blob Storage:** Secure object store hosting AES-256 encrypted PDF files in the `letters` container.

• **Azure Monitor & Log Stream:** Tracks real-time HTTP requests, diagnostics, and system health.

Slide 7: Technologies Used

Component	Technology / Service Used
Frontend	HTML5, CSS3, JavaScript (Vanilla SPA)
Backend	Python (FastAPI Framework + Uvicorn)
Cloud Platform	Microsoft Azure
Cloud Database (DBaaS)	Neon Serverless PostgreSQL / SQLite (Local)
Cloud Object Storage	Azure Blob Storage (`letters` container)
Cloud Hosting (PaaS)	Azure App Service (Linux B1 Basic Tier)
Security & Cryptography	AES-256-GCM, SHA-256 Hashing, JWT Authentication
Cloud Monitoring	Azure Monitor & Log Stream (HTTP logs, telemetry, metrics)

Slide 8: Azure components

• **Neon Database for PostgreSQL:**
• Stores user credentials, document metadata, file hashes, and access logs in a managed cloud database.
• Supports automated backups, connection pooling, and strict relational integrity.
• **Azure Blob Storage:**
• Stores encrypted PDF documents in a dedicated cloud container (`letters`).
• Enables secure, scalable file storage with AES-256 encryption at rest.
• **Azure App Service:**
• Hosts the FastAPI-based web application and serves responsive user dashboards.
• Provides secure web access globally through HTTPS with automated SSL certificates.
• **Azure Resource Group (`rg-docchain-demo`):**
• Organizes all project-related Azure resources in one centralized place.
• Simplifies resource management, access control, and subscription monitoring.
• **Azure Monitor / Log Stream:**
• Tracks application performance, container health, and incoming HTTP requests in real-time.
• Helps identify errors, latency, and system diagnostics during operations.

Slide 9: Cloud Deployment

• **Cloud Platform:** Microsoft Azure
• **Hosting Service:** Azure App Service
• **Live Application URL:** https://docchain-app.azurewebsites.net/
• **Deployment Details:** The DocChain application is accessible globally through a web browser using secure HTTPS.

Slide 10: Future Scope

• **Blockchain Integration:** Anchor document SHA-256 hashes onto public or consortium blockchain ledgers (e.g., Ethereum or Polygon) for decentralized, immutable verification.

• **Government Vault Integration:** Connect with national digital vaults such as DigiLocker for direct document synchronization and official verification.

• **Cloud Scalability & Microservices:** Decompose backend components into Docker containers managed by Azure Kubernetes Service (AKS) with Azure Functions for automated document expiration triggers.

• **DevOps Automation:** Automate application testing and continuous cloud deployment using GitHub Actions and Infrastructure as Code (ARM / Terraform).

Slide 11: Conclusion

• Implemented secure user management, document issuance, and real-time verification.

• Ensured data integrity and tamper-proofing using SHA-256 hashing and AES-256-GCM encryption.

• Integrated responsive frontend dashboards with a high-performance FastAPI backend and PostgreSQL database.

• Deployed the web application on Microsoft Azure using Azure App Service and Azure Blob Storage.

Slide 12: Thank You